DSCI 552 | Homework 6 study notes

Yash Desai | Study notes on APS classification and the ISLR exercises

How to read these notes: the code and results are followed by the reason behind each step. All four implemented models have been run. The official split has 60,000 training and 16,000 test rows. The positive class is pos=1, and probabilities of at least 0.5 are treated as positive predictions. No majority-class downsampling is used.

1(a)-(b). Data and preparation

The goal is to tell APS-related failures from failures involving other components. The training set has 59,000 negatives and 1,000 positives (59:1; 1.667% positive). The test set has 15,625 negatives and 375 positives (41.667:1; 2.344% positive). There are 170 numeric predictors. Always predicting negative would give only 2.344% test error, but it would miss every APS failure. That is why accuracy alone is not enough.

(b.i) Missing entries can be handled with a mean, median or mode; KNN; iterative or multiple imputation; missing-value indicators; or a model that handles them directly. Deleting incomplete rows could discard a lot of useful data. Here we use training-feature medians because they are practical and less sensitive to extreme values. This keeps all rows and 170 predictors. An entirely missing training column is retained with a zero fill. Median filling can still reduce variation and change correlations, so it has limitations.

Following the homework footnote, missing test values are filled with medians from the unlabeled test batch itself. Training rows have their own median imputer. CV mirrors this rule: each validation batch is imputed separately, while the scaler and model are fitted only on the training rows in that fold. No evaluation labels enter imputation, and evaluation statistics do not enter training or SMOTE. This batch-based protocol differs from the usual deployment practice of storing training medians for future individual predictions.

(b.ii) CV=s/m compares a feature's sample standard deviation with its mean. We calculate it after median imputation, before scaling, using ddof=1. A mean of zero makes the ratio undefined, so it is stored as NaN. The complete 170-feature result is in feature_CV_all_170.csv.

(b.iii)-(iv) The correlation matrix uses all 170 imputed training features. For the scatter and box plots, we select the 13 highest finite CV values because floor(sqrt(170))=13. This choice is only for the plots: every model still uses all 170 predictors. The plots use log1p values to make the skew easier to see. Every training observation is included, and box-plot outliers are shown with small markers.

A visible pattern can suggest an association, but it cannot establish statistical significance, conditional usefulness or causality. High CV means high relative variation, not necessarily strong predictive value. Formal significance claims would need a separate inferential procedure, with attention to multiple comparisons.

1(c)-(f). Models and validation

(c) Start with a random forest that makes no adjustment for imbalance: 300 bootstrap trees, sqrt(p) candidate features at each split, unrestricted depth and seed 552. The outputs show training and test confusion matrices, ROC curves, AUC and error=(FP+FN)/n. Matrix rows are actual [neg,pos]; columns are predicted [neg,pos]. For OOB error, each observation is predicted using trees that left it out of their bootstrap sample. We compare that estimate with test error. Since the imputer was fitted before the forest, OOB checks the forest with that imputer already in place, rather than cross-fitting the whole preprocessing pipeline.

(d) Ways to address imbalance include class weights, balanced bootstrap samples, oversampling, undersampling and thresholds chosen on training validation data. Here balanced_subsample recalculates inverse-frequency weights in each bootstrap sample; the other forest settings stay the same. Full-training weights would be about 0.5085 for neg and 30 for pos, a 59:1 relative ratio. Weighting makes rare-class mistakes matter more during fitting, but its effect on the test results still needs to be checked.

(e): a wording issue to keep in mind. XGBoost's gblinear booster fits one global logistic model with an L1 penalty controlled by reg_alpha. It does not fit logistic regressions at individual tree nodes. On a tree booster, reg_alpha penalizes leaf scores instead. This project uses gblinear with binary:logistic for the L1-logistic part of the task. It therefore does **not** claim to implement the literal node-wise model tree described in the question; that interpretation still needs clarification.

For the linear models, median filling is followed by log1p and standardization. All APS predictors are nonnegative. Training-only checks found numerical saturation without the log transform, so the transform is used consistently in both comparisons. The fit uses coordinate descent, cyclic feature selection, 100 rounds, learning rate 0.1 and no L2 penalty. Five shuffled, stratified folds compare positive alpha values {0.0001,0.001,0.01,0.1}, using seed 552. The lowest mean classification error wins, with AUC as tie breaker. Because the same folds select alpha, the selected CV error may be optimistic; it is not nested CV. Test error is reported separately.

(f) SMOTE belongs inside each CV training fold. Fit the training imputer and scaler, then create synthetic minority examples from the transformed training rows. Impute validation rows using their own unlabeled batch medians and apply the training-fitted scaler; never resample validation rows. Applying SMOTE before CV could let validation observations influence synthetic training points and make the score look too good. After selecting a positive alpha, refit preprocessing and SMOTE(k=5) on the full training set: 59,000 negatives and 59,000 positives, including 58,000 synthetic positives. That gives 118,000 fitting rows. Scores still use the original 60,000 training and 16,000 test observations. No threshold is chosen from test results.

The point of the comparison is to see the tradeoff, not assume that SMOTE must win. We look at missed failures, false alarms, recall, precision, ROC AUC, average precision and overall error. Scaling helps keep large-unit features from dominating SMOTE distances. Distances in many dimensions and interpolated histogram values still have limitations, and the changed training balance can affect probability calibration.

1. What the model results show

Model / split	Error	ROC AUC	Recall	Precision	AP
rf_plain/train	0.0000	1.0000	1.0000	1.0000	1.0000
rf_plain/test	0.0076	0.9944	0.7200	0.9408	0.9256
rf_weighted/train	0.0000	1.0000	1.0000	1.0000	1.0000
rf_weighted/test	0.0107	0.9931	0.5813	0.9397	0.8851
xgb_plain/train	0.0085	0.9863	0.5910	0.8504	0.7975
xgb_plain/test	0.0119	0.9916	0.5787	0.8715	0.8392
xgb_smote/train	0.0421	0.9876	0.9570	0.2781	0.7678
xgb_smote/test	0.0413	0.9905	0.9573	0.3576	0.8245

rf_plain: the OOB error is 0.00575, compared with 0.00762 on the test set. The test-minus-OOB difference is +0.00187.

rf_weighted: the OOB error is 0.00767, compared with 0.01069 on the test set. The test-minus-OOB difference is +0.00302.

xgb_plain: CV selected alpha=0.0001. Its five-fold error is 0.00908, compared with test error 0.01187. The test-minus-CV difference is +0.00279. The final model was fitted on 60,000 rows.

xgb_smote: CV selected alpha=0.0001. Its five-fold error is 0.04210, compared with test error 0.04131. The test-minus-CV difference is -0.00079. The final model was fitted on 118,000 rows.

rf_weighted versus rf_plain: test error changes by +0.00306, recall by -0.13867, and AUC by -0.00136. False negatives change from 105 to 157; false positives from 17 to 14.

xgb_smote versus xgb_plain: test error changes by +0.02944, recall by +0.37867, and AUC by -0.00114. False negatives change from 158 to 16; false positives from 32 to 645.

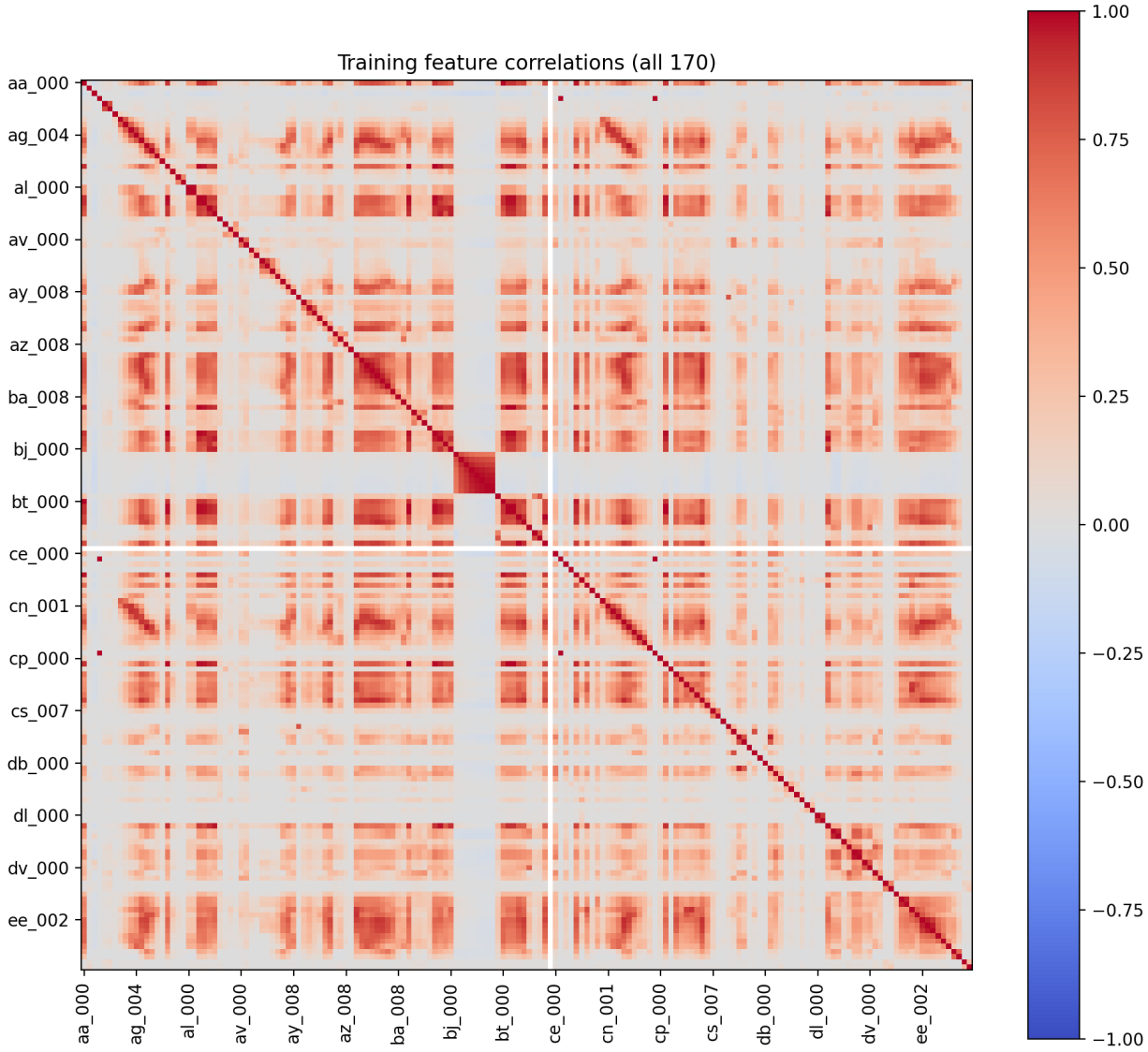
1(b). Feature results

The 13 features chosen for the plots are: cf_000, co_000, ad_000, cs_009, dh_000, dj_000, ag_000, as_000, ay_009, ak_000, az_009, ch_000, au_000.

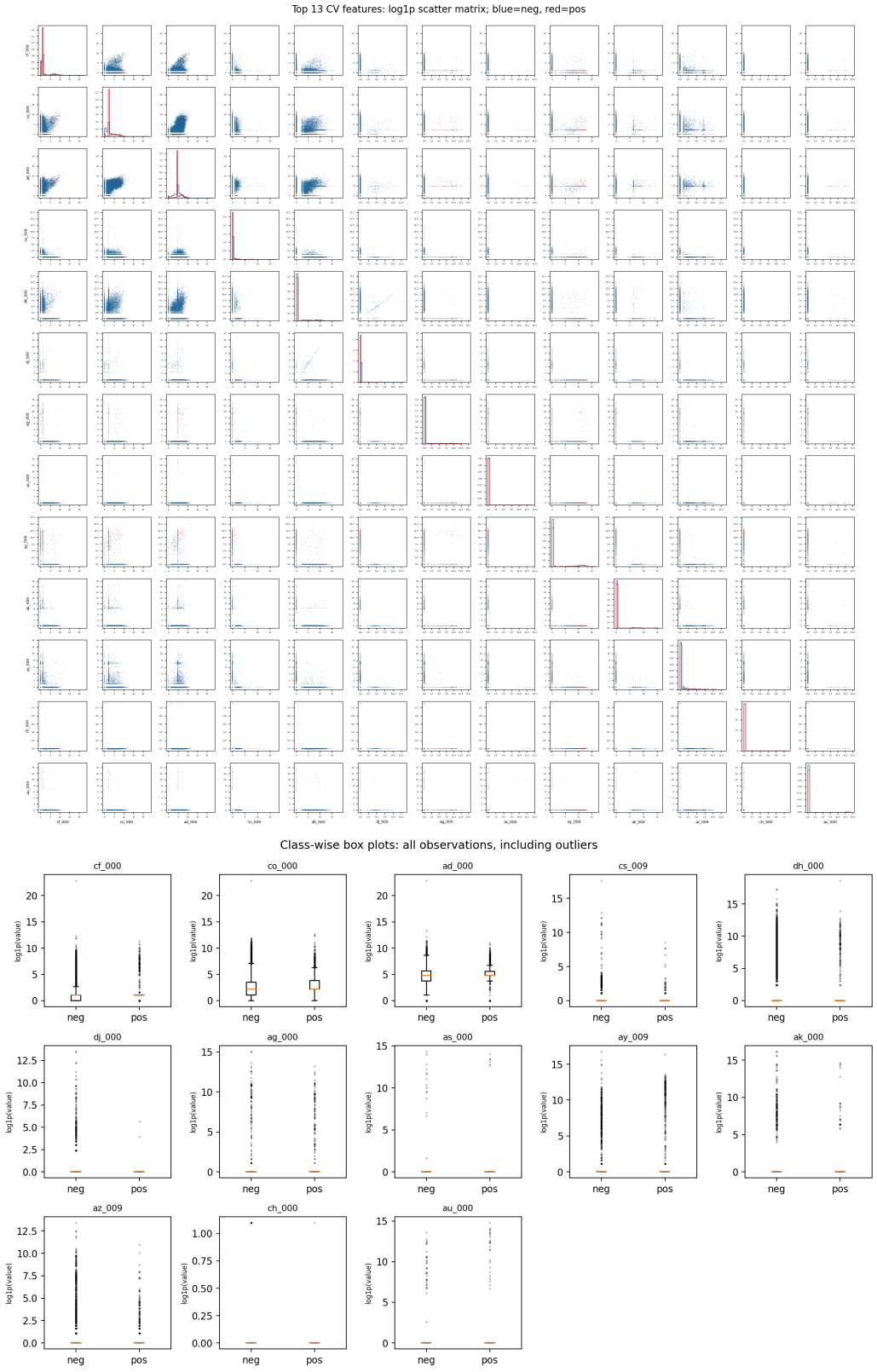
Several high-CV features have similar class medians or a median of zero in both classes. Rare extreme values account for much of their variation. The correlation matrix also shows overlapping information across predictors. Together, these plots explain why the highest-CV features should not automatically be treated as the best classifiers.

Feature	Mean	Sample SD	CV
cf_000	1.4311e+05	3.5045e+07	244.89
co_000	1.4333e+05	3.5045e+07	244.51
ad_000	1.4344e+05	3.5045e+07	244.32
cs_009	770.51	1.8333e+05	237.93
dh_000	4392.6	5.4124e+05	123.22
dj_000	37.271	4379.2	117.49
ag_000	219.16	20364	92.918
as_000	125.41	10952	87.332
ay_009	1149.6	97412	84.734
ak_000	907.66	72999	80.425
az_009	41.603	3238.3	77.839
ch_000	0.00033333	0.025818	77.454
au_000	228.16	15717	68.883

1(b.iii). Correlation matrix

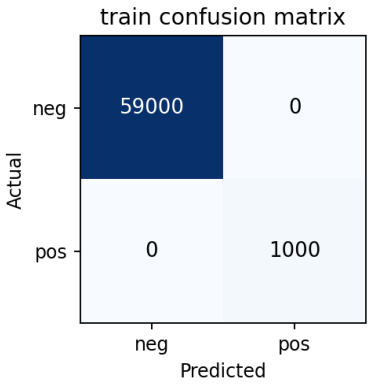


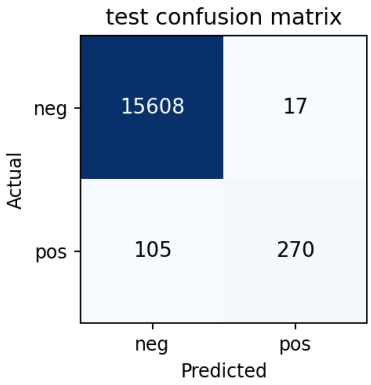
1(b.iv). Visualizations

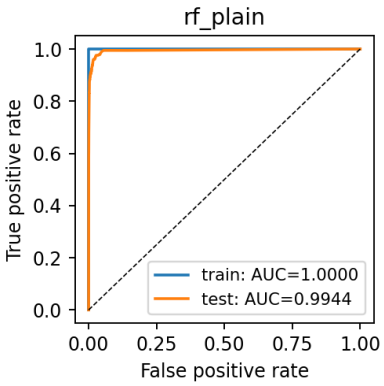


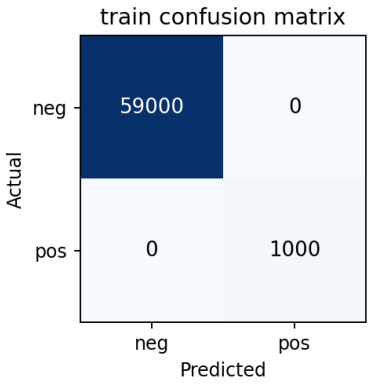
Open scatter_matrix.png separately to zoom into the 169 panels. The colors distinguish the classes. These plots help with exploration; they do not establish significance.

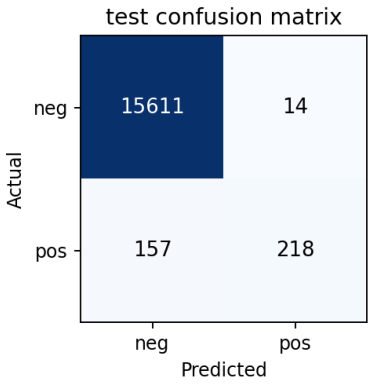
1(c)-(f). Confusion matrices and ROC

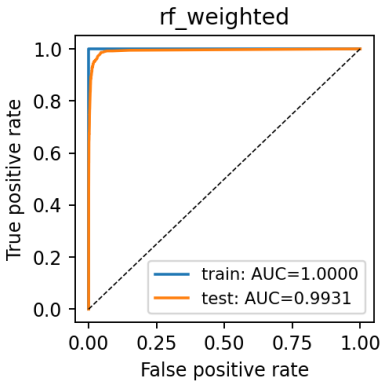


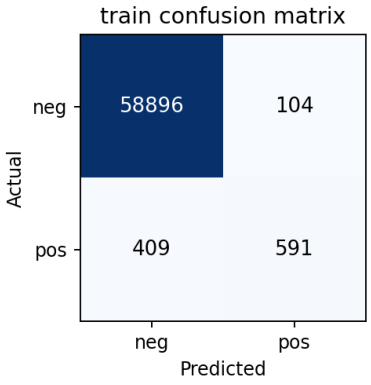


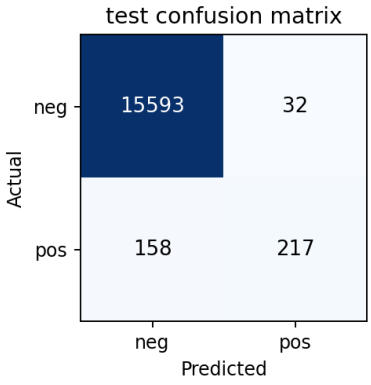


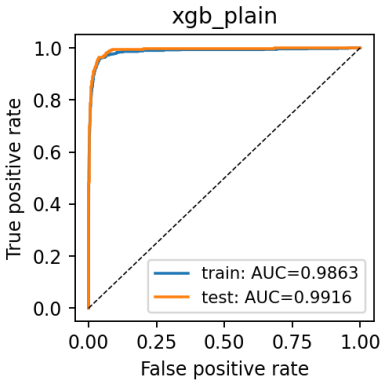


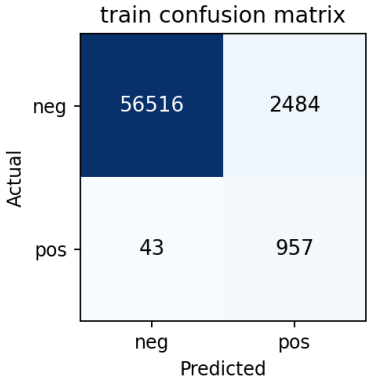


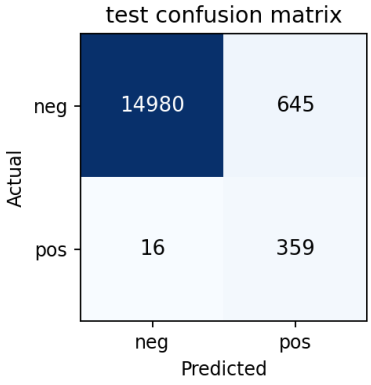


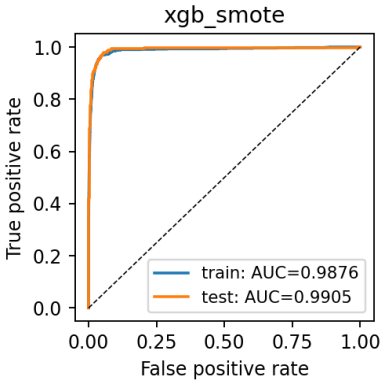












1(e)-(f). Alpha selection

Five-fold CV: plain

alpha	mean_error	sd_error	mean_auc
0.0001	0.00908333	0.000738335	0.985116
0.001	0.00988333	0.00118087	0.983324
0.01	0.0127833	0.00101824	0.965824
0.1	0.0166667	0	0.5

Five-fold CV: smote

alpha	mean_error	sd_error	mean_auc
0.0001	0.0421	0.00180701	0.984367
0.001	0.04405	0.00180624	0.984628
0.01	0.0501167	0.00117792	0.981787
0.1	0.0641333	0.00311727	0.9683

2. ISLR 6.6.3 - relaxing the lasso constraint

The useful idea here is that s controls how much freedom the lasso has. We minimize RSS while requiring sum_j |beta_j| <= s. At s=0, every slope is zero. As s gets larger, more coefficient values become possible. Once the least-squares solution fits inside the constraint, loosening it further makes no difference.

(a) Training RSS decreases or stays the same, so choice iv is the best answer. A larger s keeps all the earlier solutions available and adds new ones. The best fit therefore cannot become worse on the same training data.

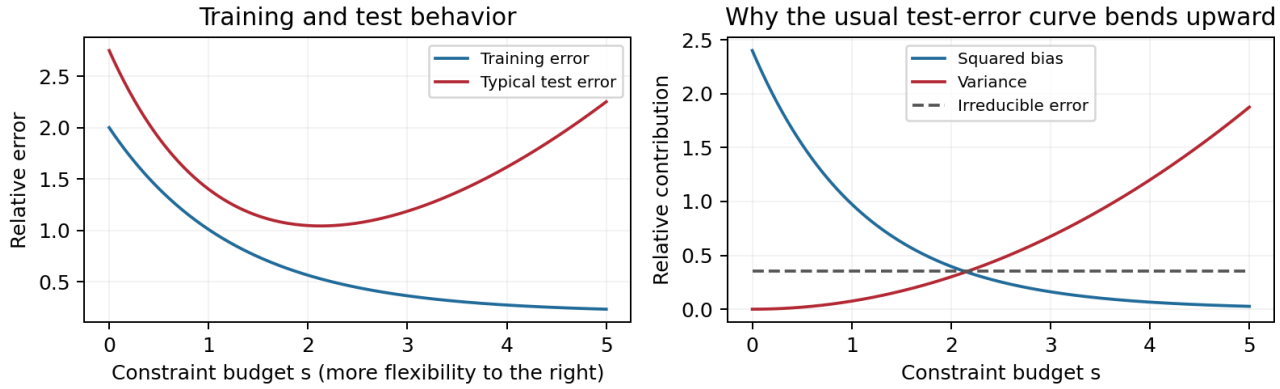
(b) Test RSS usually falls at first and then rises, which is choice ii. Initially, less shrinkage helps the model capture real patterns. With too much freedom, it can also fit noise. This is the usual bias-variance pattern, rather than a rule that every individual data set must follow.

(c) Variance generally increases, so choose iii. A more flexible fit can change more when we replace the training sample.

(d) Squared bias generally decreases, so choose iv. Relaxing the constraint reduces the restriction that was pulling the coefficients toward zero.

(e) Irreducible error stays constant, so choose v. Changing s changes the model, but it does not remove the underlying noise in the response.

Question 2: schematic learning illustration, not APS measurements



Learning illustration: these chosen curves show the usual trends as s increases. They are schematic, not measured APS results. The test-error curve reflects the balance between decreasing squared bias and increasing variance; the noise stays fixed.

3. ISLR 6.6.5 - identical predictors in ridge and lasso

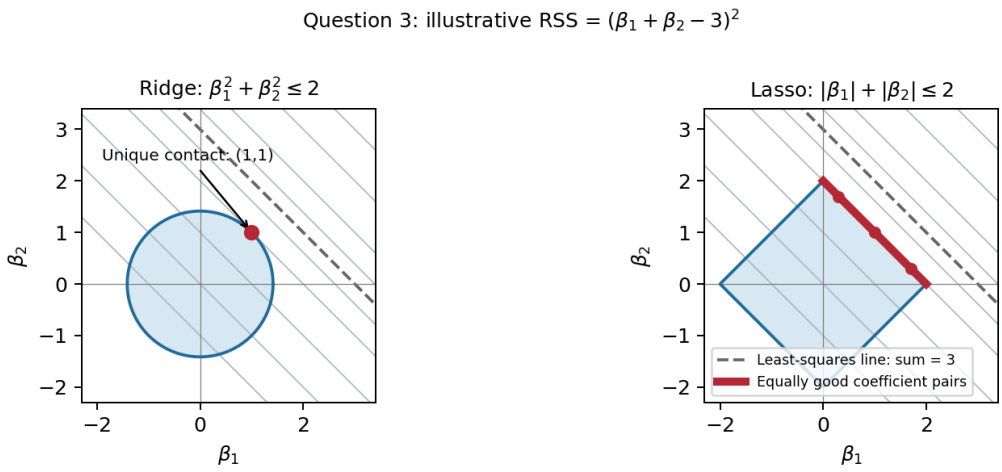
Both predictor columns contain the same information. Since the predictors and response are centered, the unpenalized intercept is zero. Write x_i for the common predictor value and t=beta_1+beta_2. Also let A=sum_i x_i^2 and B=sum_i x_i y_i. The fitted values depend on t, not on how that total is split between the two coefficients.

(a) Ridge minimizes sum_i [y_i-x_i(beta_1+beta_2)]^2 + lambda(beta_1^2+beta_2^2). The first term measures the fit, and the second discourages large coefficients. The intercept is not penalized.

(b) Differentiating gives -2 sum_i x_i[y_i-x_i(beta_1+beta_2)] + 2 lambda beta_j=0 for j=1,2. Subtracting the equations leaves 2 lambda(beta_1-beta_2)=0. For lambda>0, beta_1=beta_2=B/(2A+lambda). Ridge shares the effect equally between identical predictors. At lambda=0, equal coefficients are not required.

(c) Lasso instead minimizes sum_i [y_i-x_i(beta_1+beta_2)]^2 + lambda(|beta_1|+|beta_2|). For a fixed total t, the smallest possible penalty is lambda|t|. Any same-sign split of t achieves it, so several coefficient pairs can describe the same fit.

(d) For A>0, the best total is t*=sign(B) max(|B|-lambda/2,0)/A. If t*>0, every pair (u,t*-u) with 0<=u<=t* works equally well. If t*<0, use t*<=u<=0. This shows why lasso coefficients need not be unique even though their predictions are the same. When t*=0, the minimum-penalty pair is (0,0), so nonuniqueness is not guaranteed. In the geometric picture, an RSS contour can touch a whole edge of the L1 diamond.

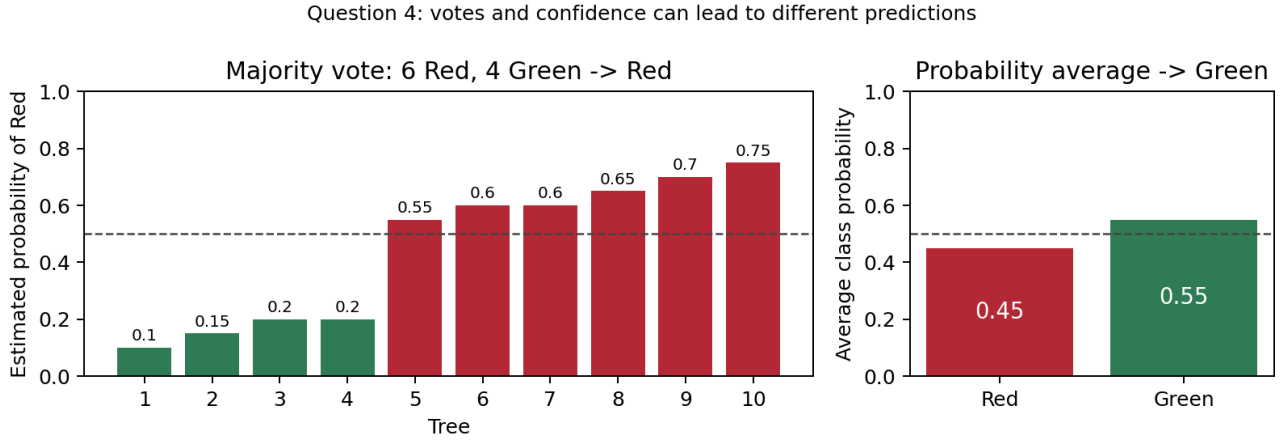


Learning illustration: with identical predictors, RSS depends on their coefficient sum. The circle has one best contact point, while the highlighted diamond edge has many equally good coefficient pairs. The two pictures use the stated constraints, not a shared penalty value.

4. ISLR 8.4.5 - combining ten tree predictions

For majority voting, each tree gets one vote: it votes Red when its Red probability is above 0.5. Six trees vote Red and four vote Green, so the final prediction is Red.

Averaging probabilities gives a different answer. The ten probabilities add to 4.5, so the average Red probability is 0.45. That makes the average Green probability 0.55, and this method predicts Green. The difference is that voting ignores how confident each tree is, while averaging keeps that information.



This plot uses the exact probabilities in the exercise. Six trees vote Red, but the average Red probability is only 0.45. That is why the two aggregation methods disagree.

5. ISLR 9.7.3 - understanding the maximal-margin classifier

(a) The seven points are (3,4), (2,2), (4,4), (1,4), (2,1), (4,3), and (4,1). Points 1-4 are Red and points 5-7 are Blue. The figure labels each point so it is easy to relate the geometry to the calculations.

(b) The optimal separating line is x_2=x_1-1/2, or 1/2-x_1+x_2=0. The nearest Red points lie on x_2=x_1, and the nearest Blue points lie on x_2=x_1-1. Putting the separator halfway between these parallel lines leaves equal room on both sides.

(c) Predict Red when 1/2-x_1+x_2>0 and Blue when it is negative. One set of coefficients is beta_0=1/2, beta_1=-1, beta_2=1. Multiplying all three by the same positive number changes the scale, but not the decision boundary.

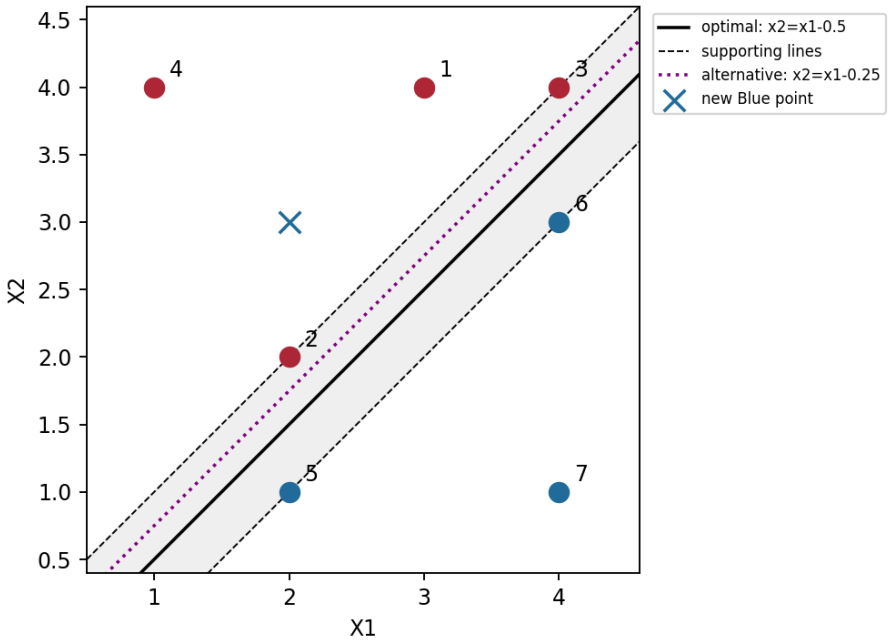
(d) The distance from the separator to either supporting line is (1/2)/sqrt(2)=1/(2 sqrt(2)), about 0.3536. This is the margin on each side; the whole strip is 1/sqrt(2), about 0.7071, wide. Divide the coefficients by sqrt(2) for a unit normal. The canonical functional-margin coefficients are (1,-2,2).

(e) Points 2, 3, 5 and 6 are the support vectors: (2,2), (4,4), (2,1), and (4,3). They sit on the edges of the margin. The closest Red and Blue convex hulls are 1/sqrt(2) apart, so no separator can leave more than half that distance on both sides.

(f) Point 7, (4,1), lies safely beyond the Blue supporting line. A small movement that keeps it on that side leaves the support vectors in place, so the maximal-margin separator stays the same.

(g) The line x_2=x_1-1/4 also separates the classes. However, its Red-side margin is only 1/(4 sqrt(2)), so it leaves less room than the optimal line. The dotted line in the figure shows this alternative.

(h) Add a Blue point at (2,3). It lies inside the Red convex hull because (2,3)=0.5(2,2)+0.25(3,4)+0.25(1,4). A half-space containing all those Red points also contains this weighted average. The two classes can therefore no longer be strictly separated by a line.



Sources and notes for studying

APS data: UCI Machine Learning Repository, APS Failure at Scania Trucks, dataset 421. https://archive.ics.uci.edu/dataset/421/aps+failure+at+scania+trucks

Imputation: scikit-learn documentation, Imputation of missing values. https://scikit-learn.org/stable/modules/impute.html

Random forest weighting: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

XGBoost parameter reference (linear booster and alpha): https://xgboost.readthedocs.io/en/release_3.2.0/parameter.html

SMOTE: Chawla et al. (2002), JAIR 16, 321-357, DOI 10.1613/jair.953. Resampling leakage: https://imbalanced-learn.org/stable/common_pitfalls.html

Textbook: James, Witten, Hastie and Tibshirani, An Introduction to Statistical Learning with Applications in R. https://www.statlearning.com/ . Chapter 6 section numbers vary across editions, and the homework uses both 6.6 and 6.8. Match these answers by the exercise number and topic: Chapter 6 Exercises 3 and 5, Chapter 8 Exercise 5, Chapter 9 Exercise 3.

Question 6 is labeled Extra Practice and is not included. Keep the model-tree wording issue from 1(e) in mind when reviewing the implemented XGBoost models.

To repeat the analysis, run python analysis.py. The full CV table and prediction arrays are saved separately so you can inspect the details without making the main report too long.

Appendix. Executable analysis code

The code below is also in analysis.py, so you can follow the calculations and run them again. The PDF layout is created by report.py. The analysis uses the full training set and all 170 predictors.

```
"""Work through APS classification, from data preparation to model comparison."""
from pathlib import Path
import io, json, urllib.request, zipfile, warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (confusion_matrix, roc_auc_score, roc_curve,
    accuracy_score, recall_score, precision_score, average_precision_score)

ROOT=Path(__file__).resolve().parent
DATA=ROOT/'data'; OUT=ROOT/'results'
SEED=552; THREADS=4; BOOST_THREADS=1
ALPHAS=[0.0001, 0.001, 0.01, 0.1]
ROUNDS=100

def load_data():
    DATA.mkdir(exist_ok=True); OUT.mkdir(exist_ok=True)
    names=['aps_failure_training_set.csv','aps_failure_test_set.csv']
    if not all((DATA/n).exists() for n in names):
        url='https://archive.ics.uci.edu/static/public/421/aps+failure+at+scania+trucks.zip'
        print('Downloading the official APS archive...',flush=True)
        try:
            with urllib.request.urlopen(url,timeout=180) as r:
                z=zipfile.ZipFile(io.BytesIO(r.read()))
            for n in names:
                matches=[m for m in z.namelist() if Path(m).name==n]
                if len(matches)!=1: raise ValueError(f'Missing or ambiguous archive member: {n}')
                (DATA/n).write_bytes(z.read(matches[0]))
        except Exception as e:
            raise RuntimeError('Download failed. Download the APS ZIP from UCI manually and put both
    CSVs in data/.') from e
    frames=[]
    for n in names:
        lines=(DATA/n).read_text().splitlines()
        start=next(i for i,l in enumerate(lines) if l.startswith('class,'))
        frames.append(pd.read_csv(io.StringIO('\n'.join(lines[start:])),na_values='na'))
    tr,te=frames
    assert tr.shape==(60000,171) and te.shape==(16000,171), (tr.shape,te.shape)
    assert tr.columns.equals(te.columns)
    assert set(tr['class'])=={'neg','pos'}
    X=tr.drop(columns='class'); T=te.drop(columns='class')
    y=tr['class'].map({'neg':0,'pos':1}).to_numpy()
    yt=te['class'].map({'neg':0,'pos':1}).to_numpy()
    assert y.sum()==1000 and yt.sum()==375
    return X,T,y,yt

def evaluation_impute(frame):
    """Follow the homework footnote: use this unlabeled evaluation batch's medians."""
    return SimpleImputer(strategy='median', keep_empty_features=True).fit_transform(frame)

def metrics(y,p):
    if not np.isfinite(p).all() or np.any((p<0)|(p>1)):
        raise ValueError('Predicted probabilities must be finite and in [0,1].')
    pred=(p>=.5).astype(int)
    return dict(error=float(1-accuracy_score(y,pred)),auc=float(roc_auc_score(y,p)),
      recall=float(recall_score(y,pred)),precision=float(precision_score(y,pred,zero_division=0)),
      ap=float(average_precision_score(y,p)),cm=confusion_matrix(y,pred,labels=[0,1]).tolist())

def plot_probabilities(name,y,yt,ps):
```

```
    fig,ax=plt.subplots(1,3,figsize=(9,3.1))
    for side,truth,p,a in zip(['train','test'],[y,yt],ps,ax[:2]):
        cm=confusion_matrix(truth,p>=.5,labels=[0,1]); a.imshow(cm,cmap='Blues')
        for (i,j),v in np.ndenumerate(cm):
    a.text(j,i,str(v),ha='center',va='center',fontsize=11,color='white' if v>cm.max()/2 else 'black')
        a.set(xticks=[0,1],yticks=[0,1],xticklabels=['neg','pos'],yticklabels=['neg','pos'],
          xlabel='Predicted',ylabel='Actual',title=side+' confusion matrix')
    for s,truth,p in zip(['train','test'],[y,yt],ps):
        fpr,tpr,_=roc_curve(truth,p); ax[2].plot(fpr,tpr,label=f'{s}:
    AUC={roc_auc_score(truth,p):.4f}')
    ax[2].plot([0,1],[0,1],'k--',lw=.7); ax[2].set(xlabel='False positive rate',ylabel='True positive
    rate',title=name)
    ax[2].legend(fontsize=9); fig.tight_layout(); fig.savefig(OUT/(name+'.png'),dpi=170);
    plt.close(fig)

def evaluate(name, model, X,T,y,yt, results):
    ps=[model.predict_proba(a)[:,1] for a in (X,T)]
    results[name]={s:metrics(b,p) for s,b,p in zip(['train','test'],[y,yt],ps)}
    np.savez_compressed(OUT/(name+'_predictions.npz'),train=ps[0],test=ps[1])
    plot_probabilities(name,y,yt,ps)
    print(name,results[name],flush=True)

def booster(alpha):
    if alpha <= 0:
        raise ValueError("Use a positive alpha for the L1-penalized models.")
    from xgboost import XGBClassifier
    return XGBClassifier(booster='gblinear',objective='binary:logistic',
       reg_alpha=alpha,reg_lambda=0,n_estimators=ROUNDS,learning_rate=.1,
       updater='coord_descent',feature_selector='cyclic',n_jobs=BOOST_THREADS,
       random_state=SEED,eval_metric='logloss')

def cv_linear(X,y,smote):
    from imblearn.over_sampling import SMOTE
    # Fit the model and scaler on training rows. Impute validation using its own unlabeled batch, as
    the footnote asks.
    folds=list(StratifiedKFold(5,shuffle=True,random_state=SEED).split(X,y))
    errors=np.zeros((len(ALPHAS),5)); aucs=np.zeros_like(errors)
    oof=np.zeros((len(ALPHAS),len(y)),dtype=np.float32)
    for k,(a,b) in enumerate(folds):
        imp=SimpleImputer(strategy='median',keep_empty_features=True)
        scaler=StandardScaler()
        xa=scaler.fit_transform(np.log1p(imp.fit_transform(X.iloc[a]))).astype('float32')
        xb=scaler.transform(np.log1p(evaluation_impute(X.iloc[b]))).astype('float32')
        ya=y[a]
        if smote: xa,ya=SMOTE(random_state=SEED+k,k_neighbors=5).fit_resample(xa,ya)
        for j,alpha in enumerate(ALPHAS):
            model=booster(alpha).fit(xa,ya)
            p=model.predict_proba(xb)[:,1]; oof[j,b]=p
            print(f'SMOTE={smote}, fold={k+1}, alpha={alpha}: fitted',flush=True)
            errors[j,k]=np.mean((p>=.5)!=y[b]); aucs[j,k]=roc_auc_score(y[b],p)
        print(f'SMOTE={smote}: finished fold {k+1}/5',flush=True)
    table=pd.DataFrame({'alpha':ALPHAS,'mean_error':errors.mean(1),
       'sd_error':errors.std(1,ddof=1),'mean_auc':aucs.mean(1)})
    # Choose the lowest CV error. If errors tie, prefer the higher AUC.
    best=int(table.sort_values(['mean_error','mean_auc'],ascending=[True,False]).index[0])
    tag='smote' if smote else 'plain'
    table.to_csv(OUT/f'cv_{tag}.csv',index=False)
    np.savez_compressed(OUT/f'oof_{tag}.npz',probability=oof[best],truth=y)
    return ALPHAS[best], float(errors[best].mean())

def exploratory(X,T,y,yt):
    imp=SimpleImputer(strategy='median',keep_empty_features=True)
    A=pd.DataFrame(imp.fit_transform(X),columns=X.columns)
    B=evaluation_impute(T)
    means=A.mean(); cv=A.std(ddof=1).div(means.where(means!=0))
    pd.DataFrame({'mean':means,'sample_sd':A.std(ddof=1),'CV':cv,
      'missing_fraction':X.isna().mean()}).to_csv(OUT/'feature_CV_all_170.csv')
    top=cv.replace([np.inf,-np.inf],np.nan).dropna().nlargest(int(np.sqrt(170))).index.tolist()
    assert len(top)==13
    corr=A.corr()
    corr.to_csv(OUT/'correlations_all_170.csv')
    fig,ax=plt.subplots(figsize=(10,9)); im=ax.imshow(corr,vmin=-1,vmax=1,cmap='coolwarm')
```

```
    ax.set(xticks=np.arange(0,170,10),yticks=np.arange(0,170,10),
      xticklabels=X.columns[::10],yticklabels=X.columns[::10],title='Training feature correlations
    (all 170)')
    plt.setp(ax.get_xticklabels(),rotation=90); fig.colorbar(im,ax=ax); fig.tight_layout()
    fig.savefig(OUT/'correlation.png',dpi=200); plt.close(fig)
    # Plot every training observation in the 13-by-13 matrix. The log1p transform
    # makes the skewed values easier to see; CV is still calculated in original units.
    Z=np.log1p(A[top]); n=len(top)
    fig,axes=plt.subplots(n,n,figsize=(22,22))
    colors=np.where(y==1,'#b62832','#246a9a')
    for i in range(n):
      for j in range(n):
        ax=axes[i,j]
        if i==j:
          for c,color in [(0,'#246a9a'),(1,'#b62832')]:
            ax.hist(Z.iloc[y==c,i],bins=25,density=True,histtype='step',color=color)
        else: ax.scatter(Z.iloc[:,j],Z.iloc[:,i],c=colors,s=.4,alpha=.12,rasterized=True)
        ax.tick_params(labelsize=4)
        if i==n-1: ax.set_xlabel(top[j],fontsize=7)
        if j==0: ax.set_ylabel(top[i],fontsize=7)
    fig.suptitle('Top 13 CV features: log1p scatter matrix; blue=neg, red=pos',fontsize=18)
    fig.tight_layout(rect=[0,0,1,.98]); fig.savefig(OUT/'scatter_matrix.png',dpi=130); plt.close(fig)
    fig,axes=plt.subplots(3,5,figsize=(13,8))
    for ax,col in zip(axes.flat,top):
        ax.boxplot([Z.loc[y==0,col],Z.loc[y==1,col]],tick_labels=['neg','pos'],showfliers=True,flierpr
    ops={'marker':'.','markersize':2,'alpha':.3})
        ax.set_title(col,fontsize=9); ax.set_ylabel('log1p(value)',fontsize=7)
    for ax in list(axes.flat)[13:]: ax.axis('off')
    fig.suptitle('Class-wise box plots: all observations, including outliers')
    fig.tight_layout(); fig.savefig(OUT/'boxplots.png',dpi=170); plt.close(fig)
    return A.to_numpy(dtype='float32'), B.astype('float32'), {'top13':top,
      'train_neg':int((y==0).sum()),'train_pos':int(y.sum()),
      'test_neg':int((yt==0).sum()),'test_pos':int(yt.sum()),
      'missing_fraction':float(X.isna().to_numpy().mean())}

def main():
    from imblearn.over_sampling import SMOTE
    X,T,y,yt=load_data(); A,B,info=exploratory(X,T,y,yt)
    results={'data':info,'models':{}}
    for tag,weight in [('rf_plain',None),('rf_weighted','balanced_subsample')]:
        model=RandomForestClassifier(n_estimators=300,max_features='sqrt',bootstrap=True,
          oob_score=True,class_weight=weight,n_jobs=THREADS,random_state=SEED)
        model.fit(A,y); evaluate(tag,model,A,B,y,yt,results['models'])
        results['models'][tag]['oob_error']=float(1-model.oob_score_)
    for smote in [False,True]:
        tag='xgb_smote' if smote else 'xgb_plain'
        alpha,cv_error=cv_linear(X,y,smote)
        imp=SimpleImputer(strategy='median',keep_empty_features=True); scaler=StandardScaler()
        xa=scaler.fit_transform(np.log1p(imp.fit_transform(X))).astype('float32')
        xb=scaler.transform(np.log1p(evaluation_impute(T))).astype('float32')
        fitx,fity=(SMOTE(random_state=SEED,k_neighbors=5).fit_resample(xa,y) if smote else (xa,y))
        model=booster(alpha).fit(fitx,fity)
        # Score the original training rows so the comparison uses the real class balance.
        evaluate(tag,model,xa,xb,y,yt,results['models'])
        results['models'][tag].update(alpha=alpha,cv_error=cv_error,fit_rows=int(len(fity)))
    (OUT/'results.json').write_text(json.dumps(results,indent=2))
    import report
    report.build(results)
    print('Finished. Open results/HW6_Results.pdf',flush=True)
    return results

if __name__=='__main__': main()
```